

Panel Econometrics Tutorial 871

Introduction to Application Programming Interfaces (API)

World Bank Data API as an Example

Precious Nhamo

Stellenbosch University

Week of 20 April 2026

Contents

1	Introduction: APIs and Reproducible Empirical Work	3
2	What is an API?	3
3	Setup and Package Management	4
4	Finding Indicator Codes	4
4.1	Using <code>wbstats</code> (Recommended)	4
4.2	Using WDI (Classic)	5
5	Understanding Indicator Code Suffixes	5
6	Country Codes and Time Granularity	6
6.1	ISO Country Codes: ISO2 vs. ISO3	6
6.2	Time Granularity in API Calls	6
7	Downloading Data: Core Workflow	6
7.1	Using <code>wbstats</code>	6
7.2	Using WDI (Classic)	7
7.3	Filtering by Region or Income Group	7
7.4	Understanding the Returned Dataframe	7
8	Reading Stata Files into R	8
8.1	Basic Loading	8
8.2	Working with Stata Labels	8
8.3	Stripping Stata Attributes for a Plain Dataframe	8
9	Rebasing and Deflating: Essential Transformations	9
9.1	Mathematical Formulation	9
9.2	Rebasing an Index in R	9
9.3	Deflating Nominal GDP	10
9.4	Using the GDP Deflator	10
10	A Reusable Wrapper Function	11
10.1	Example Usage	12
11	Quick-Reference Indicator Dictionary	12

12 Replication Resources and Best Practices	12
13 Useful Links	14
14 Exercise: From World Bank to Another API – Create, Visualise, and Publish	14
Conclusion	16

1 Introduction: APIs and Reproducible Empirical Work

Empirical economics increasingly demands transparent, reproducible workflows. Leading journals—including the *Quarterly Journal of Economics*, the *Review of Economic Studies*, and the *American Economic Review*—require authors to provide replication packages containing data and code. We are currently in a golden age of replication packages; using them to your advantage can accelerate your research and sharpen your technical skills. Platforms such as the **Harvard Dataverse** and **Zenodo** host replication materials for thousands of papers, allowing you to verify results, learn from exemplary code, and adapt methods for your own projects.

This tutorial focuses on the World Bank’s API as a foundational tool for obtaining macroeconomic and development indicators. By automating data acquisition, you ensure that your analysis is:

- **Reproducible:** Your script documents exactly which data were used, eliminating ambiguity about versions or download dates.
- **Updatable:** When the World Bank revises historical GDP figures, re-running your script integrates the revisions automatically.
- **Scalable:** Fetching 50 countries $\times$ 10 indicators $\times$ 60 years becomes trivial with one API call.
- **Transparent:** Metadata about units, currencies, and observation statuses are embedded in the returned data.

We will also cover how to read Stata `.dta` files (common in replication packages) and how to perform basic data transformations—rebasings and deflating—that are essential for any empirical macroeconomic analysis. The goal is to equip you with a reusable template that you can apply immediately to your own research and coursework.

2 What is an API?

API stands for **Application Programming Interface**. It is a set of rules that allows one piece of software to communicate with another. A useful analogy is to think of an API as a waiter in a restaurant:

You (R script)	Place an order — write one line of code requesting data.
The API (waiter)	Carries your request to the World Bank server.
World Bank server (kitchen)	Looks up the data and prepares it for delivery.
The API (waiter)	Returns exactly what you asked for, clean and structured.

Manual workflow (without an API):

1. Navigate to the World Bank DataBank website.
2. Manually select indicators, countries, and years.
3. Download a CSV file.
4. Clean and reshape the data in Excel or R.
5. Repeat every time the data are updated.

With an API, one line of R code fetches the latest data directly into your environment, preserving a complete audit trail.

World Bank API Coverage

- Over 1,600 indicators across national accounts, prices, labour, health, education, and governance.
- 217 economies (countries and regional aggregates).
- More than 50 years of historical data for many series.
- Free access — no login or API key required.

3 Setup and Package Management

We will use two main packages to interface with the World Bank API: `wbstats` (modern, tidyverse-friendly) and `WDI` (classic, widely cited). In addition, we load `tidyverse` for data manipulation and `haven` for reading Stata files. A best practice is to use the `pacman` package to manage library loading and installation in one step.

```

1 # Install pacman if missing, otherwise load it
2 if (!require("pacman", quietly = TRUE)) {
3   install.packages("pacman")
4   library(pacman)
5 }
6
7 # p_load installs (if necessary) and loads multiple packages
8 p_load(wbstats, WDI, tidyverse, haven)
9
10 # Verify that the packages are ready
11 library(help = "WDI")           # lists functions in the WDI package
12 wbstats::wb_cachelist           # glimpse of cached World Bank metadata
13 WDI::WDI_data                  # explore WDI data structures

```

Listing 1: Package installation and loading with pacman

Why pacman? It streamlines package management, especially when sharing code with collaborators who may not have all dependencies installed. It also reduces the clutter of multiple `library()` calls.

4 Finding Indicator Codes

Each World Bank indicator has a unique alphanumeric code (e.g., `NY.GDP.PCAP.KD`). The code is the key that the API uses to retrieve the correct series. Both `wbstats` and `WDI` provide search functions.

4.1 Using wbstats (Recommended)

```

1 # Search for indicators containing "GDP"
2 gdp_indicators <- wb_search("GDP")
3 head(gdp_indicators)
4
5 # Narrow search to "GDP per capita"
6 gdp_pc <- wb_search("GDP per capita")
7 View(gdp_pc)    # interactive table viewer in RStudio
8
9 # Important columns:
10 #   indicator_id  -> code for wb_data()
11 #   indicator     -> human-readable description
12 #   unit          -> measurement unit (e.g., "constant 2015 US$")

```

Listing 2: Searching for indicators with `wb_search`

4.2 Using WDI (Classic)

```
1 # Search for indicators containing "inflation"
2 WDIsearch("inflation")
```

Listing 3: Searching with `WDIsearch`

5 Understanding Indicator Code Suffixes

The suffix of an indicator code encodes crucial information about the unit, adjustment method, and base year. Misinterpreting suffixes is a common source of errors in empirical work. Table 1 provides a comprehensive list.

Table 1: Comprehensive List of Common World Bank Indicator Suffixes

Suffix	Meaning and Appropriate Use
.CD	Current USD (nominal). Affected by both inflation and exchange rate movements. Use for headline figures, single-period snapshots, or when nominal values are explicitly required.
.KD	Constant USD (real, inflation-adjusted). Base year is usually 2015. Use for comparing growth rates over time within a country. The base year is indicated in the indicator description.
.PP.CD	Purchasing Power Parity (PPP) adjusted, expressed in current international dollars. Corrects for cross-country differences in price levels. Use for comparing living standards <i>across</i> countries at a point in time.
.PP.KD	PPP-adjusted, expressed in constant international dollars. Combines inflation adjustment with PPP conversion. Use for comparing growth in living standards across countries over time.
.ZS	Percentage share (ratio). Already unit-free (e.g., trade as % of GDP, unemployment rate). Values are typically between 0 and 100.
.GD	Share of GDP. Similar to .ZS but explicitly relative to GDP.
.LN	Natural logarithm of the underlying series. Rarely used directly; often employed in internal calculations.
.MA	Moving average or smoothed series. Indicates that the value has been smoothed over time.
.DT	Decadal data (e.g., population census every 10 years). Check the indicator description for the exact frequency.
Country Codes	
.AR	Annual Report data. May differ slightly from standard series due to reporting conventions.
.EST	Estimated value (not directly reported by the country). Used by World Bank staff to fill gaps.

Economist's rule of thumb:

- **Cross-country level comparisons** → use .PP.CD or .PP.KD (PPP-adjusted).

- **Growth over time within a country** → use `.KD` (constant/real).
- **Policy reports in nominal USD** → use `.CD` (current/nominal).

6 Country Codes and Time Granularity

6.1 ISO Country Codes: ISO2 vs. ISO3

The World Bank API accepts both two-letter (ISO2) and three-letter (ISO3) country codes. It is important to understand the distinction and to be consistent in your work.

- **ISO2** (e.g., ZA, US, NG): Two-letter codes defined by ISO 3166-1 alpha-2. They are concise and often used in plots or quick identification.
- **ISO3** (e.g., ZAF, USA, NGA): Three-letter codes defined by ISO 3166-1 alpha-3. They are more informative and are the standard for merging with other international datasets (e.g., from the UN, IMF, or Penn World Tables).

Recommendation: Always use `iso3c` when merging datasets. The `wbstats` package returns both `iso2c` and `iso3c` columns by default.

6.2 Time Granularity in API Calls

Most World Bank indicators are reported annually. However, some series (e.g., exchange rates, commodity prices) are available at higher frequencies. The `wbstats` package automatically handles the format, but when constructing raw API calls, you must specify dates correctly:

- **Annual data:** Use YYYY format (e.g., 2020).
- **Quarterly data:** Use YYYYQ[1-4] format (e.g., 2020Q1).
- **Monthly data:** Use YYYYMM[01-12] format (e.g., 2020M01).

The `wb_data()` function accepts `start_date` and `end_date` as numeric (for annual) or character (for quarterly/monthly) and handles the conversion internally. For advanced users, knowing the underlying format is helpful when debugging or making direct HTTP requests.

7 Downloading Data: Core Workflow

7.1 Using `wbstats`

```

1 # Define indicators using a named vector (names become column names)
2 my_indicators <- c(
3   gdp_per_capita = "NY.GDP.PCAP.KD", # constant 2015 USD
4   inflation      = "FP.CPI.TOTL.ZG", # annual CPI inflation (%)
5   trade_openness = "NE.TRD.GNFS.ZS", # trade as % of GDP
6   school_enroll  = "SE.SEC.ENRR"    # secondary gross enrolment ratio
7 )
8
9 # Specify countries (ISO2 or ISO3 codes)
10 my_countries <- c("ZA", "NG", "KE", "GH", "EG")
11
12 # Download data for 2000-2023
13 africa_data <- wb_data(
14   indicator = my_indicators,
15   country   = my_countries,
16   start_date = 2000,

```

```

17   end_date   = 2023
18 )
19
20 # Inspect the structure
21 head(africa_data)
22 glimpse(africa_data) # displays column types and units

```

Listing 4: Fetching data with wb_data

7.2 Using WDI (Classic)

```

1 wdi_data <- WDI(
2   indicator = c(
3     gdp_per_capita = "NY.GDP.PCAP.KD",
4     inflation      = "FP.CPI.TOTL.ZG"
5   ),
6   country   = c("ZA", "NG", "KE", "GH", "EG"),
7   start     = 2000,
8   end       = 2023
9 )
10 head(wdi_data)

```

Listing 5: Fetching data with WDI

7.3 Filtering by Region or Income Group

```

1 # Retrieve country metadata
2 country_info <- wb_countries()
3
4 # Extract ISO3 codes for Sub-Saharan Africa
5 ssa_countries <- country_info %>%
6   filter(region == "Sub-Saharan Africa") %>%
7   pull(iso3c)
8
9 # Download GDP per capita for all SSA countries
10 ssa_data <- wb_data(
11   indicator = c(gdp_pc = "NY.GDP.PCAP.KD"),
12   country   = ssa_countries,
13   start_date = 2000,
14   end_date   = 2022
15 )
16 # Approximately 48 countries    23 years = ~1,104 observations

```

Listing 6: Downloading data for all Sub-Saharan African countries

7.4 Understanding the Returned Dataframe

wbstats returns a tidy dataframe where each row corresponds to one country-year observation. Key columns include:

```

1 # Summarise missing values across all columns
2 africa_data %>%
3   summarise(across(everything(), ~ sum(is.na(.))))
4
5 # Note: Missing data often reflects non-reporting, which may correlate with
6 # institutional quality. Consider the implications for your analysis.

```

Listing 7: Checking for missing values

Table 2: Key Columns in the Returned Dataframe

Column	Description
iso2c / iso3c	ISO country codes. Use <code>iso3c</code> for merging with other datasets.
country	Full country name as labeled by the World Bank.
date	Year as a numeric value (or character for quarterly/monthly).
[indicator name]	The data values in the unit indicated by the suffix.
obs_status	Data quality flag: ‘estimate’, ‘provisional’, or ‘actual’. Always check before analysis.
footnote	Source notes and important caveats from the World Bank.

8 Reading Stata Files into R

Replication packages from journals like the *Quarterly Journal of Economics* or *Review of Economic Studies* frequently distribute data in Stata’s `.dta` format. The `haven` package reads these files seamlessly, preserving variable and value labels as attributes.

8.1 Basic Loading

```

1 library(haven)
2
3 # Load a .dta file
4 macro_data <- read_dta("data/macro_panel.dta")
5
6 # For files saved in newer Stata versions, specify encoding
7 macro_data <- read_dta("data/macro_panel.dta", encoding = "UTF-8")

```

Listing 8: Loading a Stata dataset

8.2 Working with Stata Labels

```

1 # Retrieve variable label (description of the variable)
2 attr(macro_data$gdp, "label")
3
4 # Retrieve value labels (mappings for categorical variables)
5 attr(macro_data$region_code, "labels")
6
7 # Convert labelled variables to R factors
8 macro_data <- macro_data %>%
9   mutate(region = as_factor(region_code))

```

Listing 9: Accessing and converting Stata labels

8.3 Stripping Stata Attributes for a Plain Dataframe

```

1 # Create a clean R dataframe without Stata attributes
2 clean_df <- macro_data %>%
3   zap_labels() %>% # remove value label attributes
4   zap_formats() %>% # remove display format attributes
5   as.data.frame()

```

Listing 10: Removing Stata-specific attributes

Best Practices for Replication Packages

1. **Read the README:** It explains the data structure, variable definitions, and the order in which scripts should be executed.
2. **Trace variable creation:** If a Stata `.do` file references a variable that does not exist in the distributed data, search earlier `.do` files to find where it is generated.
3. **Check for auxiliary data:** Some replication packages require downloading additional datasets from external sources. The README should note this.
4. **Consider using here:** The `here` package helps manage file paths, making your code more portable across machines.

9 Rebasing and Deflating: Essential Transformations

Economic series are rarely ready for analysis in their raw downloaded form. Two fundamental operations are rebasing index numbers and deflating nominal values to obtain real quantities.

9.1 Mathematical Formulation

Let I_t be an index series with base year b_0 (i.e., $I_{b_0} = 100$). To rebase the index to a new base year b , we compute:

$$I_t^{(new)} = \frac{I_t}{I_b} \times 100 \quad (1)$$

This operation changes the scale but preserves growth rates between any two periods:

$$\frac{I_t^{(new)}}{I_{t-1}^{(new)}} = \frac{I_t/I_b}{I_{t-1}/I_b} = \frac{I_t}{I_{t-1}}.$$

Deflating converts a nominal series N_t into a real series R_t using a price index P_t (with $P_{base} = 100$):

$$R_t = \frac{N_t}{P_t} \times 100 \quad (2)$$

Equivalently, we express real values in terms of the base year's purchasing power. The real growth rate is then:

$$g_t = \frac{R_t - R_{t-1}}{R_{t-1}}. \quad (3)$$

9.2 Rebasing an Index in R

```

1 # Download CPI data (default base 2010)
2 cpi_data <- wb_data(
3   indicator = c(cpi = "FP.CPI.TOTL"),
4   country   = c("ZA", "NG", "KE"),
5   start_date = 2000, end_date = 2023
6 )
7
8 # Extract base year (2015) values for each country
9 base_values <- cpi_data %>%
10   filter(date == 2015) %>%
11   select(iso3c, cpi_base = cpi)
12
13 # Apply rebasing formula (Equation 1)
14 cpi_rebased <- cpi_data %>%

```

```

15 left_join(base_values, by = "iso3c") %>%
16 mutate(cpi_2015 = (cpi / cpi_base) * 100) %>%
17 select(-cpi_base)
18
19 # Verify: 2015 values should all be exactly 100
20 cpi_rebased %>%
21 filter(date == 2015) %>%
22 select(country, cpi_2015)

```

Listing 11: Rebasing CPI from 2010=100 to 2015=100

9.3 Deflating Nominal GDP

```

1 # Download nominal GDP and CPI
2 gdp_cpi <- wb_data(
3   indicator = c(
4     gdp_nominal = "NY.GDP.MKTP.CD", # GDP, current USD
5     cpi          = "FP.CPI.TOTL"    # CPI, 2010=100
6   ),
7   country      = c("ZA", "NG", "KE"),
8   start_date   = 2000, end_date   = 2022
9 )
10
11 # Compute real GDP (base 2010) using Equation (2)
12 gdp_real <- gdp_cpi %>%
13   mutate(gdp_real_2010 = gdp_nominal / (cpi / 100))
14
15 # Sanity check: nominal and real should be equal in the base year
16 gdp_real %>%
17   filter(date == 2010) %>%
18   select(country, gdp_nominal, gdp_real_2010)

```

Listing 12: Deflating nominal GDP with CPI

9.4 Using the GDP Deflator

For aggregate output, investment, or government expenditure, the GDP deflator is more appropriate than the CPI. The deflator is defined as:

$$\text{GDPDef}_t = \frac{\text{Nominal GDP}_t}{\text{Real GDP}_t} \times 100. \quad (4)$$

```

1 # Download nominal GDP and GDP deflator
2 gdp_deflator_data <- wb_data(
3   indicator = c(
4     gdp_nominal = "NY.GDP.MKTP.CD",
5     gdp_deflator = "NY.GDP.DEFL.ZS" # GDP deflator, 2010=100
6   ),
7   country      = "ZA",
8   start_date   = 2000, end_date   = 2022
9 )
10
11 # Compute real GDP
12 gdp_deflator_data <- gdp_deflator_data %>%
13   mutate(gdp_real = gdp_nominal / (gdp_deflator / 100))

```

Listing 13: Deflating with the GDP deflator

CPI vs. GDP Deflator: Which to Use?

- **CPI:** Reflects prices of a fixed basket of consumer goods. Use for deflating household consumption or income series.
- **GDP Deflator:** Reflects prices of all domestically produced goods and services. Use for deflating aggregate output, investment, or government spending.

For growth regressions, the pre-deflated constant-USD series (.KD) is often the safest and most convenient choice.

10 A Reusable Wrapper Function

Encapsulating the data-fetching logic in a function promotes code reuse and reduces the chance of errors. The following function downloads data, attaches country metadata, and optionally rebases index series.

```

1  #' Fetch World Bank data with optional rebasing
2  #'
3  #' @param indicators Named vector of indicator codes, e.g., c(gdp = "NY.GDP.
4  #' @param countries Character vector of ISO3 country codes, or "all"
5  #' @param start_year Numeric start year
6  #' @param end_year Numeric end year
7  #' @param rebase_year If not NULL, rebase all index series to this year
8  #' @return A tidy data frame with country metadata and (optionally) rebased
9  #'         columns
10 fetch_wb_data <- function(indicators,
11                             countries = "all",
12                             start_year = 2000,
13                             end_year = 2023,
14                             rebase_year = NULL) {
15   # 1. Download the data
16   df <- wb_data(
17     indicator = indicators,
18     country = countries,
19     start_date = start_year,
20     end_date = end_year,
21     return_wide = TRUE
22   )
23
24   # 2. Merge country metadata (region, income level, lending type)
25   meta <- wb_countries() %>%
26     select(iso3c, region, income_level, lending_type)
27   df <- df %>% left_join(meta, by = "iso3c")
28
29   # 3. Optionally rebase index series
30   if (!is.null(rebase_year)) {
31     base <- df %>%
32       filter(date == rebase_year) %>%
33       select(iso3c, where(is.numeric)) %>%
34       rename_with(~ paste0(.x, "_base"), -iso3c)
35
36     df <- df %>% left_join(base, by = "iso3c")
37
38     for (ind in names(indicators)) {
39       base_col <- paste0(ind, "_base")
40       if (base_col %in% names(df)) {
41         new_col <- paste0(ind, "_rebased_", rebase_year)
42         df[[new_col]] <- (df[[ind]] / df[[base_col]]) * 100
43       }
44     }
45   }

```

```

45   df <- df %>% select(-ends_with("_base"))
46 }
47
48 return(df)
49 }

```

Listing 14: Reusable wrapper for World Bank data

10.1 Example Usage

```

1  # Basic call: GDP per capita, inflation, and trade openness
2  result <- fetch_wb_data(
3    indicators = c(
4      gdp_pc    = "NY.GDP.PCAP.KD",
5      inflation = "FP.CPI.TOTL.ZG",
6      trade     = "NE.TRD.GNFS.ZS"
7    ),
8    countries  = c("ZA", "NG", "KE", "GH"),
9    start_year = 2005,
10   end_year   = 2022
11 )
12
13 # With rebasing: rebase CPI to 2015 = 100
14 result_rebased <- fetch_wb_data(
15   indicators = c(cpi = "FP.CPI.TOTL"),
16   countries  = c("ZA", "NG", "KE"),
17   start_year = 2000,
18   end_year   = 2023,
19   rebase_year = 2015
20 )

```

Listing 15: Using the wrapper function

11 Quick-Reference Indicator Dictionary

Table 3: National Accounts

Indicator Code	Description
NY.GDP.MKTP.CD	GDP, current USD (nominal)
NY.GDP.MKTP.KD	GDP, constant 2015 USD (real)
NY.GDP.PCAP.CD	GDP per capita, current USD
NY.GDP.PCAP.KD	GDP per capita, constant 2015 USD
NY.GDP.PCAP.PP.CD	GDP per capita, PPP (current international \$)
NY.GDP.PCAP.PP.KD	GDP per capita, PPP (constant 2017 international \$)
NY.GDP.DEFL.ZS	GDP deflator (index, 2010=100)
NE.CON.TOTL.ZS	Final consumption expenditure (% of GDP)
NE.GDI.TOTL.ZS	Gross capital formation (% of GDP)
NE.TRD.GNFS.ZS	Trade (exports + imports) as % of GDP

12 Replication Resources and Best Practices

The advice provided to graduate students often emphasises the importance of engaging with replication packages from top journals. Below are actionable steps to integrate these resources

Table 4: Prices and Monetary

Indicator Code	Description
FP.CPI.TOTL	Consumer Price Index (index, 2010=100)
FP.CPI.TOTL.ZG	CPI inflation, annual %
FR.INR.RINR	Real interest rate (%)
PA.NUS.FCRF	Official exchange rate (LCU per USD, period avg.)

Table 5: Labour and Poverty

Indicator Code	Description
SL.UEM.TOTL.ZS	Unemployment, total (% of labour force)
SL.TLF.CACT.ZS	Labour force participation rate, total (%)
SI.POV.DDAY	Poverty headcount at \$2.15/day (% of pop.)
SI.POV.GINI	Gini index (0 = perfect equality, 100 = perfect inequality)

Table 6: Human Capital and Demographics

Indicator Code	Description
SP.POP.TOTL	Population, total
SP.DYN.LE00.IN	Life expectancy at birth, total (years)
SE.PRM.ENRR	Primary school enrolment, gross %
SE.SEC.ENRR	Secondary school enrolment, gross %
SH.XPD.CHEX.GD.ZS	Current health expenditure (% of GDP)

into your workflow.

Key Platforms

- **Harvard Dataverse:** Hosts replication data for the *Quarterly Journal of Economics*, *Journal of Political Economy*, *Review of Economics and Statistics*, and many others.
- **Zenodo:** Used by the *Review of Economic Studies (ReStud)* for replication packages.
- **Authors' Personal Websites:** Researchers like Daron Acemoglu, Raj Chetty, and Jan De Loecker often post replication code and data.

Recommended Replication Packages to Study

1. **Baqae and Farhi (2020, QJE):** “Productivity and Misallocation in General Equilibrium.” The replication package includes Compustat data processing scripts. This is an excellent exercise in reading and adapting Stata or MATLAB code to R.
2. **Raval (2023, ReStud):** “Testing the Production Approach to Markup Estimation.” A clean example of modern empirical industrial organization.
3. **Autor et al. (2020):** Illustrates the long lag between working paper (2017) and publication (2020). Checking NBER working paper series can give you a head start on the literature frontier.

Workflow for Using a Replication Package

1. **Download** the package from the journal's Dataverse or Zenodo repository.
2. **Read the README** file carefully. It should describe the required software, data sources, and the order of script execution.
3. **Set up a clean R project** and copy the data files into a `data/` subfolder.
4. **Translate data-loading steps first.** Use `haven` for `.dta` files, `readr` for `.csv`, and `readxl` for Excel files.
5. **Run the original code** (in Stata or MATLAB) side-by-side with your R translation to verify that you reproduce the reported results exactly.
6. **Document any discrepancies** and note differences in default settings (e.g., Stata's `reg` vs. R's `lm` handling of missing values).

Staying Current with the Literature Frontier

- NBER, CEPR, IZA working paper series.
- Job market papers of PhD candidates from top departments (MIT, Harvard, Chicago, Stanford, etc.).
- Conference programs from the NBER Summer Institute, Econometric Society meetings.

13 Useful Links

- World Bank API Documentation: <https://datahelpdesk.worldbank.org/knowledgebase/articles/889386-developer-information-overview>
- `wbstats` GitHub repository: <https://github.com/gshs-ornl/wbstats>
- WDI package documentation: <https://cran.r-project.org/web/packages/WDI/index.html>
- Harvard Dataverse: <https://dataverse.harvard.edu/>
- *Review of Economic Studies* replication packages on Zenodo: <https://zenodo.org/communities/restud/>
- Country ISO codes reference: https://wits.worldbank.org/wits/wits/witshelp/content/codes/country_codes.htm

14 Exercise: From World Bank to Another API – Create, Visualise, and Publish

The skills you have acquired in this tutorial are not limited to the World Bank API. Most economic data providers offer RESTful APIs that follow similar principles. This exercise asks you to apply your knowledge to a **different** API, produce two meaningful visualisations, and publish your work on GitHub.

Choose One of the Following APIs

1. UNdata API – <https://data.un.org/Host.aspx?Content=API>
2. OECD API – <https://data.oecd.org/api/>
3. FRED (Federal Reserve Economic Data) – <https://fred.stlouisfed.org/docs/api/fred/>
4. IMF Data API – <http://data.imf.org/>
5. COVID-19 Data (JHU CSSE) – <https://github.com/CSSEGISandData/COVID-19>

Tasks

1. **Set up a new R project** and initialise a Git repository.
2. **Write an R script** that:
 - (a) Fetches data from your chosen API (use appropriate R packages if available, or construct HTTP requests with `httr` or `jsonlite`).
 - (b) Cleans and reshapes the data into a tidy format.
 - (c) Creates **two meaningful plots** using `ggplot2`. The plots should convey an economic insight (e.g., a time-series comparison across countries, a scatter plot with a regression line, or a thematic map).
3. **Write a README.md** that includes:
 - A brief description of the data source and the indicators used.
 - Instructions for reproducing your analysis (how to install required packages and run the script).
 - A short interpretation of each plot.
4. **Push your project to GitHub** and ensure the repository is public.
5. **Submit the GitHub repository link** as proof of completion.

Example: Using the FRED API

The `fredr` package provides an R interface to the FRED API. A sample workflow is shown below:

```

1 # Install and load fredr
2 # install.packages("fredr")
3 library(fredr)
4 library(tidyverse)
5
6 # Set your FRED API key (obtain from https://fred.stlouisfed.org/docs/api/api_
   key.html)
7 fredr_set_key("your_api_key_here")
8
9 # Fetch US unemployment rate (UNRATE) and CPI (CPIAUCSL)
10 unrate <- fredr(series_id = "UNRATE", observation_start = as.Date("2000-01-01")
   )
11 cpi <- fredr(series_id = "CPIAUCSL", observation_start = as.Date("2000-01-01
   "))
12
13 # Plot 1: Unemployment rate over time
14 p1 <- ggplot(unrate, aes(x = date, y = value)) +

```

```

15   geom_line(color = "steelblue") +
16   labs(title = "US Unemployment Rate", y = "Percent", x = "Year") +
17   theme_minimal()
18
19   # Plot 2: Inflation rate (year-over-year percentage change in CPI)
20   cpi_yoy <- cpi %>%
21     arrange(date) %>%
22     mutate(inflation = (value / lag(value, 12) - 1) * 100)
23
24   p2 <- ggplot(cpi_yoy, aes(x = date, y = inflation)) +
25     geom_line(color = "darkred") +
26     labs(title = "US CPI Inflation (YoY %)", y = "Percent", x = "Year") +
27     theme_minimal()
28
29   # Save plots (optional)
30   ggsave("unemployment.png", p1, width = 8, height = 5)
31   ggsave("inflation.png", p2, width = 8, height = 5)

```

Listing 16: Example: Fetching and plotting FRED data

Evaluation Criteria

- Code is well-commented and follows the tidyverse style guide.
- The README is clear and enables another student to reproduce your work.
- Plots are appropriately labelled and convey a clear message.
- The repository is properly structured (e.g., `data/` folder for raw data, `R/` for scripts, `figures/` for output).

Conclusion

This tutorial has provided a rigorous, reproducible framework for accessing and preparing World Bank data in R. By mastering API calls, understanding indicator suffixes, reading Stata files, and performing rebasing/deflation, you are now equipped to handle the data needs of most empirical macroeconomics and development projects. The reusable wrapper function serves as a template that you can extend to include other APIs (e.g., UNdata, OECD, or IMF) as your research demands.

Remember the core advice: “Most researchers are educators—they like to see students applying themselves.” Use replication packages not only to verify results but to learn from the coding and methodological choices of leading scholars. Show up, engage with the material, and build a portfolio of clean, transparent code on platforms like GitHub. These habits will distinguish you in both academic and industry settings.